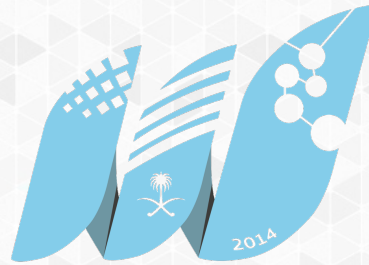


COLLAGE OF COMPUTER
SCIENCE & ENGINEERING

UNIVERSITY OF JEDDAH



جامعة جدة
University of Jeddah

كلية علوم
وهندسة الحاسب

جامعة جدة

JS: JavaScript For Web Development 2

CCSW 321 (Web Development)

What will be covered

- Asynchronous JavaScript.
- JSON/XML.
- APIs and Webservices.
- The AJAX Technique.

Async JavaScript

- **Asynchronous programming** is a programming pattern that allows code to **run independently** of the main program flow.
- In web development, we often deal with tasks that take time, such as fetching data from a server or handling user input.
- JavaScript is **single-threaded**. It uses an **event loop** to manage asynchronous tasks. The event loop continuously checks the message queue for tasks to execute. When a task is found, it's executed, and the loop continues.
- To work with **asynchronous operations**, JavaScript provides several techniques, including **callbacks**, **promises**, and **async/await**.

Async JavaScript

When is Asynchronous code usually used?

When performing data-intensive or time-consuming operations, e.g.,:

- **Fetching Data from APIs:** When you need to retrieve data from external APIs (e.g., weather data, social media feeds, or financial information), it's essential to use asynchronous JavaScript to avoid blocking the main thread while waiting for the response.
- **File Uploads and Downloads:** When users upload or download files, these operations are typically time-consuming and should be performed asynchronously. Progress bars are often used to provide feedback during these tasks.
- **Real-Time Chat Applications:** In chat applications, messages need to be sent and received in real-time. Asynchronous JavaScript is used to handle message sending and retrieval without disrupting the user's interaction with the app.

Async JavaScript - Callbacks

What are **Callbacks**?

- **Callbacks** functions are a traditional way to handle asynchronous operations.
- **Callbacks** are passed as arguments to other functions and are executed later, often after some asynchronous operation has completed.

Examples where **Callbacks** May be used:

- **setTimeout**: Schedules a callback function to run after a **specified delay**.
- **setInterval**: Repeatedly calls a callback function at a **specified interval**.
- **addEventListener**: Registers a callback function to be executed when a specified event occurs on an element.

Async JavaScript - Callbacks

- For example, let's look at `setTimeout(callback, delay)`.

```
setTimeout(myFunction, 3000);  
  
function myFunction() {  
  console.log("Hello CCSW321 Class");  
}
```

- myFunction is used as a callback.
- myFunction is passed to setTimeout() as an argument.
- 3000 is the number of milliseconds before time-out, so myFunction() will be called after 3 seconds.

Async JavaScript - Promises

What are Promises?

- **Promises** provide a cleaner way to work with asynchronous code.
- A promise represents a value that may not be available yet but will be in the future. It can be in one of **three states**: **pending**, **resolved** or **rejected**.
- **Pending State:** A Promise starts in the pending state. This is its initial state before any operation is completed. During this state, the Promise is neither fulfilled nor rejected. It's waiting for an asynchronous operation to complete.
- **Resolved State:** Once the asynchronous operation is successful, the Promise transitions to the fulfilled (resolved) state. This transition is triggered by calling the resolve function, which is provided as the first argument to the Promise constructor or as an argument to the resolve function inside a promise executor function.

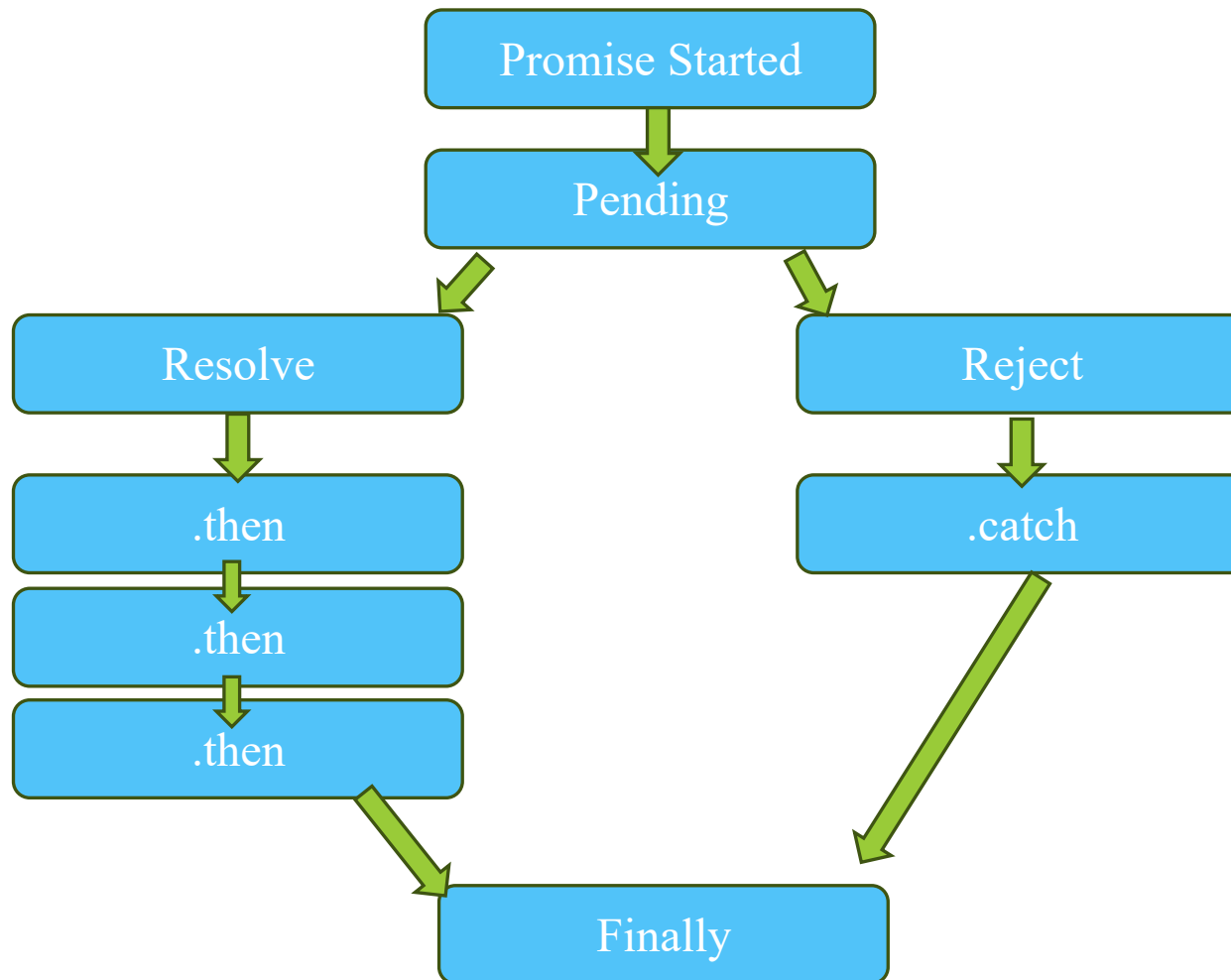
Async JavaScript - Promises

What are Promises?

- The Promise now holds the result value that was passed to resolve. Any “.then” methods associated with the Promise are executed, and the result is passed to them.
- **Rejected State:** If the asynchronous operation encounters an error or fails, the Promise transitions to the rejected state. This transition is triggered by calling the reject function. The Promise now holds the reason for the rejection, typically an error message or an exception. Any “.catch” methods associated with the Promise are executed, and the reason for rejection is passed to them.
- Promises allow you to chain “.then” and “.catch” methods to define how to handle the result or error when the Promise transitions to the fulfilled or rejected state, respectively.

Async JavaScript - Promises

- **Promises cycle:**



Async JavaScript - Promises

- Promises syntax:

```
let myPromise = new Promise(function(resolve, reject) {  
  // "Producing Code" (May take some time)  
  
  resolve("myresult"); // when successful  
  reject(); // when error  
});  
  
// What to do after task is complete?  
// encapsulate code in .then() as below  
myPromise.then(  
  function(value) { /* code if successful */},  
  function(error) { /* code if some error */}  
).then(value => { /* in case more actions are needed */})  
.catch((err)=>{ /* global error catching */})  
.finally(()=>{ /* to run any closure code */})
```

Async JavaScript – Async/Await

- **Async/await** is a more recent addition to JavaScript (ES2017) that **simplifies asynchronous code** further. It allows you to write asynchronous code in a more synchronous and readable manner.
- It uses the “**async**” **keyword** to define asynchronous functions and the “**await**” **keyword** to pause execution until a promise is resolved.

Syntax

```
//this function will run asynchronously
async function asyncFunc(){
    //executing method1() and method2() which is
    //"Producing Code" (May take some time)
    await method1();//will wait for this to complete
    await method2();//will wait for this to complete
}

//calling method with proper error handling
try {asyncFunc()} catch (error) {/* code if some error */}
```

Promise vs. Async/await

- Async/Await use Promises under the hood. They simply offer a more readable version.

```
async function
  let data = await getData();
  let formattedData = await formatData(data);
  return formattedData;
```



Equivalent

```
promise
  .then(function(result) {
    return(getData(result));
  })
  .then(function(newResult) {
    return(formatData(newResult);
  });
```

Async JavaScript - Example

Code

```
let classes = [{name:"Web Development", code:"CCSW321"}]
//printing to the web page
function printClasses(){
    let output = "";
    classes.forEach(function(element){
        output += "<li>" + element.code+": " + element.name + "</li>";
    })
    document.getElementById("content").innerHTML = output
}
//adding a new class
function addClass(myclass){
    classes.push(myclass)
}
//adding a new class after some processing
function addClassAfterProcessing(myclass){
    //doing some processing that needs sometime, then adding the class
    setTimeout(()=>{
        classes.push(myclass)
    }, 1000)//1s delay
}
//calling the functions
addClass({name:"Advanced Web Development", code:"CCSW444"})
addClassAfterProcessing({name:"Front-end Development", code:"CCSW555"})
printClasses()
```

Result

Classes:

- CCSW321: Web Development
- CCSW444: Advanced Web Development

How to solve this?

Why was this new class not added?

Because it executed after the printClasses() function got invoked

Async JavaScript - Example

- Solution using the Callback approach

```
function addClassAfterProcessingWithCallback(myclass, callback){  
    //doing some processing that needs sometime, then adding the class  
    setTimeout(()=>{  
        classes.push(myclass)  
        callback();  
    }, 1000)//1s delay  
}  
addClassAfterProcessingWithCallback({name:"Front-end Development",  
code:"CCSW555"}, printClasses)
```

Classes:

- CCSW321: Web Development
- CCSW444: Advanced Web Development
- CCSW555: Front-end Development

Async JavaScript - Example

- Solution using the Promise approach

```
function addClassAfterProcessingWithPromise(myclass){
  let mypromise = new Promise(function(resolve,reject){
    //doing some processing that needs sometime, then adding the class
    setTimeout(()=>{
      classes.push(myclass)
      resolve();
    }, 1000)//1s delay
  })
  return mypromise;
}

addClassAfterProcessingWithPromise({name:"Front-end Development",
code:"CCSW555"}).then(()=>{printClasses();})
```

Classes:

- CCSW321: Web Development
- CCSW444: Advanced Web Development
- CCSW555: Front-end Development

Async JavaScript - Example

- Solution using the async/await approach

```
async function addClassAfterProcessingWithAsync(myclass){
  await new Promise(function(resolve, reject){
    //doing some processing that needs sometime, then adding the class
    setTimeout(()=>{
      classes.push(myclass)
      resolve();
    }, 1000)//1s delay
  })
  printClasses();
}
addClassAfterProcessingWithAsync({name:"Front-end Development", code:"CCSW555"})
```

Classes:

- CCSW321: Web Development
- CCSW444: Advanced Web Development
- CCSW555: Front-end Development

APIs, Webservices, and AJAX

- An **API** is a set of rules and protocols that allows different software applications to **communicate with each other**.
- APIs define the **methods, data formats**, and conventions for **how applications should interact**.
- They **enable integration between different systems and services**.
- **Web services** are a **specific type of API** that is accessed **over a network**, typically **using HTTP** as the communication protocol.
- They allow applications to **interact with each other** over the internet and **exchange data in a standardized format**.

APIs, Webservices, and AJAX

- Web services use standard protocols and formats such as HTTP, XML, and JSON, making it easier for different systems to understand and work with the data.
- **Examples of web services:**
 - Weather information.
 - Shipping calculations.
 - Google Web APIs gives query access to Google's web search.

APIs, Webservices, and AJAX

Types of Web Services

- **SOAP** (Simple Object Access Protocol)
- **REST** (Representational State Transfer).
- **JSON-RPC** (JavaScript Object Notation Remote Procedure Call).

APIs, Webservices, and AJAX

SOAP (Simple Object Access Protocol):

- **SOAP** is a protocol for exchanging structured information in web services using **XML**.
- It provides a standardized way of communication between different systems and platforms.
- SOAP messages are **XML-based** and can be sent over various protocols such as HTTP, SMTP, or others.
- **Example:** A web service that **provides real-time stock market data** using SOAP protocol.

APIs, Webservices, and AJAX

REST (Representational State Transfer):

- REST is a **lightweight architectural style** for designing networked applications, commonly used in web services.
- REST used **XML and JSON**, however, **JSON is more commonly used** due to its lightweight and human-readable nature.
- REST is **simpler and more lightweight** compared to SOAP. It uses simple and intuitive URL patterns to access and manipulate resources.
- **Example:** A weather API that allows retrieving weather information for a specific location using RESTful endpoints like GET /weather/{city}.

APIs, Webservices, and AJAX

JSON-RPC (JavaScript Object Notation Remote Procedure Call):

- JSON-RPC is a **protocol** for **remote procedure calls** using **JSON** as the data format.
- It enables **invoking methods or functions on remote servers** and receiving the results as JSON.
- JSON-RPC requests and responses are sent over **HTTP** or other transport protocols.
- **Example:** An online **payment gateway API** that allows making payment requests and receiving transaction details using JSON-RPC.

APIs, Webservices, and AJAX

Communicating with a Webservice.

- Now, let's explore **two** widely used **data formats** for representing and transmitting data: **JSON** (JavaScript Object Notation) and **XML** (eXtensible Markup Language).
- JSON and XML play a crucial role in web development, API communication, and data interchange.
- These data formats provide a standardized way to structure, store, and transmit data between applications and systems.
- JSON and XML have their own characteristics, syntax, and best use cases.

APIs, Webservices, and AJAX

XML (eXtensible Markup Language).

- XML is a **markup language** that uses tags to **define and structure data**.
- It is designed to be both human-readable and machine-readable.
- XML documents follow a **hierarchical structure**, with nested elements and attributes.
- It allows for the creation of custom tags, making it highly extensible and flexible.
- XML is **commonly used** for **data storage**, **configuration files**, and **data exchange** between different systems.

APIs, Webservices, and AJAX

JSON (JavaScript Object Notation)

- JSON is a **lightweight data interchange format** inspired by JavaScript object syntax.
- It represents data as **key-value pairs** or as ordered lists of values.
- JSON is human-readable and easy to understand.
- It has a **simpler syntax compared to XML**, making it more concise and lightweight.
- JSON is **commonly used for data transmission** between a server and a web application, as well as storing configuration settings and representing complex data structures.

APIs, Webservices, and AJAX

XML Vs. JSON

- **Syntax:** XML **uses tags** to define elements, attributes, and structure, while JSON **utilizes key-value pairs** and arrays.
- **Readability:** XML is more verbose and has a larger file size due to its use of tags and closing elements, while JSON is more compact and easier to read.
- **Parsing and Processing:** XML **requires an XML parser** to parse and process the data, while JSON can **be parsed directly using JavaScript's built-in methods**, making it faster and more efficient.
- **Data Types:** XML supports a wider range of data types, including complex types and mixed content, while JSON **has a limited set of data types** such as strings, numbers, booleans, arrays, and objects.

APIs, Webservices, and AJAX

JSON

```
{
  "paintings": [
    {
      "id": 290,
      "title": "Balcony",
      "artist": {
        "name": "Manet",
        "nationality": "France"
      },
      "year": 1868,
      "medium": "Oil on canvas"
    },
    {
      "id": 192,
      "title": "The Kiss",
      "artist": {
        "name": "Klimt",
        "nationality": "Austria"
      },
      "year": 1907,
      "medium": "Oil and gold on canvas"
    },
    {
      "id": 139,
      "title": "The Oath of the Horatii",
      "artist": {
        "name": "David",
        "nationality": "France"
      },
      "year": 1784,
      "medium": "Oil on canvas"
    }
  ]
}
```

LISTING 17.12 JSON representation of XML data from Listing 17.1

XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<art>
  <painting id="290">
    <title>Balcony</title>
    <artist>
      <name>Manet</name>
      <nationality>France</nationality>
    </artist>
    <year>1868</year>
    <medium>Oil on canvas</medium>
  </painting>
  <painting id="192">
    <title>The Kiss</title>
    <artist>
      <name>Klimt</name>
      <nationality>Austria</nationality>
    </artist>
    <year>1907</year>
    <medium>Oil and gold on canvas</medium>
  </painting>
  <painting id="139">
    <title>The Oath of the Horatii</title>
    <artist>
      <name>David</name>
      <nationality>France</nationality>
    </artist>
    <year>1784</year>
    <medium>Oil on canvas</medium>
  </painting>
</art>
```

LISTING 17.1 Sample XML document

APIs, Webservices, and AJAX

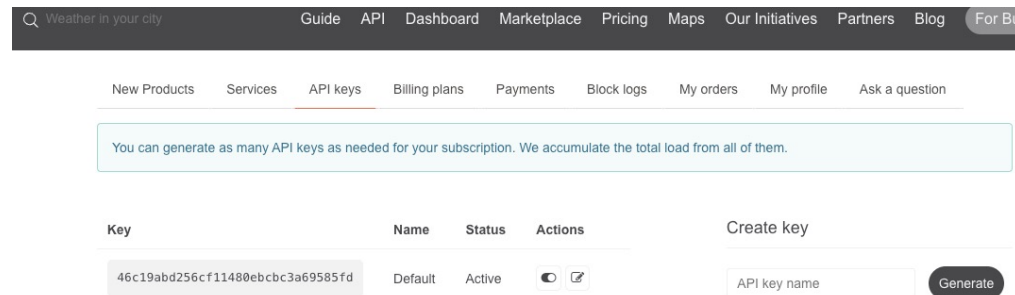
Let's see how this is done.

- **Task:** Create a page that displays the weather information for a given location.
- **Issue:** How can we know the latest weather information? We use a weather API!

- **Solution:**



- Visit the weather API and **reads its documentation**: <https://openweathermap.org/api>
- **Create an account** to get an **API key**. This will allow us to use the service.



APIs, Webservices, and AJAX

Let's see how this is done.

- **Solution:**

- In the documentation (<https://openweathermap.org/api/one-call-3>), we are shown an example for how to use the API along with the parameters.
- This is a **RESTful API** that uses **JSON**.
- We can invoke it using a **GET** request and we should expect a **JSON** response.
- We can provide parameters as needed with the request such as **lat**, **lon**, **units**, **lang**, etc.

Current and forecast weather data

To get access to current weather, minute forecast for 1 hour, hourly forecast for 48 hours, daily forecast for 8 days and government weather alerts, please use this section of the documentation.

If you are interested in **historical weather data**, please read the ["Historical weather data" section](#).

How to make an API call

API call

```
https://api.openweathermap.org/data/3.0/onecall?lat={lat}&lon={lon}&exclude={part}&appid={API key}
```

Parameters

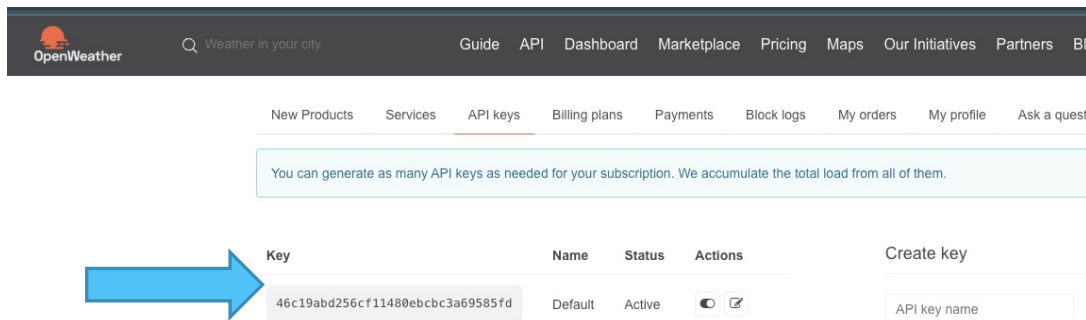
lat, lon	required	Geographical coordinates (latitude, longitude). If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API .
appid	required	Your unique API key (you can always find it on your account page under the "API key" tab)
exclude	optional	By using this parameter you can exclude some parts of the weather data from the API response. It should be a comma-delimited list (without spaces). Available values: - current - minutely - hourly - daily - alerts
units	optional	Units of measurement. <code>standard</code> , <code>metric</code> and <code>imperial</code> units are available. If you do not use the <code>units</code> parameter, <code>standard</code> units will be applied by default. Learn more
lang	optional	You can use the <code>lang</code> parameter to get the output in your language. Learn more

APIs, Webservices, and AJAX

Let's see how this is done.

- **Solution:**

- Let's try to get the weather information for Jeddah through this API:
- <https://api.openweathermap.org/data/2.5/weather?units=metric&lat=21.48&lon=39.19&appid=46c19abd256cf11480ebcbc3a69585fd>
- Change the highlighted text to your own API key, which you can find in your account.



The screenshot shows the OpenWeather API dashboard. The 'API keys' tab is selected. A table lists the API keys, with the first key highlighted. A blue arrow points to the 'Key' column.

Key	Name	Status	Actions
46c19abd256cf11480ebcbc3a69585fd	Default	Active	 

```
{
  - coord: {
    lon: 39.192,
    lat: 21.4849
  },
  - weather: [
    - {
      id: 804,
      main: "Clouds",
      description: "overcast clouds",
      icon: "04n"
    }
  ],
  base: "stations",
  - main: {
    temp: 30.95,
    feels_like: 32.93,
    temp_min: 30.95,
    temp_max: 30.95,
    pressure: 1006,
    humidity: 52,
    sea_level: 1006,
    grnd_level: 1004
  },
  visibility: 10000,
  - wind: {
    speed: 0.54,
    deg: 154,
    gust: 1.34
  },
  - clouds: {
    all: 87
  },
  dt: 1684173100,
  - sys: {
    type: 1,
    id: 7410,
    country: "SA",
    sunrise: 1684118706,
    sunset: 1684166053
  },
  timezone: 10800,
  id: 105343,
  name: "Jeddah",
  cod: 200
}
```

APIs, Webservices, and AJAX

Let's see how this is done.

- **Issue:** How can we send this request using JavaScript?
- **Solution:** We use AJAX!
- **AJAX (Asynchronous JavaScript and XML)** is a technique used to **send and retrieve data** from a server asynchronously **without reloading the entire web page**. It allows for **dynamic updates** and improves the user experience by providing seamless interaction with the server.

APIs, Webservices, and AJAX

Let's see how this is done.

There are multiple ways to perform AJAX (Asynchronous JavaScript and XML):

- **Native JavaScript:** Using the **XMLHttpRequest object**, you can create AJAX requests and handle the response manually.
- **Fetch API:** Introduced in modern browsers, the Fetch API provides a simpler and more powerful way to make AJAX requests and handle responses using promises.
- **jQuery AJAX functions:** jQuery provides a set of AJAX functions that abstract the underlying details and provide a simplified API for making AJAX requests.

APIs, Webservices, and AJAX

Let's see how this is done.

- How to perform AJAX using **native JS**?

```
// Preparing AJAX Request
const xhr = new XMLHttpRequest();
let endpoint = `Add API Endpoint here`;
```

```
//Prepare GET Request
xhr.open("GET", endpoint);
xhr.onload = function () {
  if (xhr.status === 200) {
    const response = JSON.parse(xhr.responseText);
    //do something with the response
  } else {
    //show error message "Failed to fetch weather data."
  }
};
xhr.onerror = function () {
  //show error message "An error occurred while fetching weather data."
};
```

```
//send GET request
xhr.send();
```

1. Create an XMLHttpRequest object
2. Use open() to specify HTTP method and API endpoint.
3. Use the onload event to define a function that will be called when the request is successfully completed.
4. Set up onerror to specify what to do in case an error was returned
5. Send the request.

APIs, Webservices, and AJAX

Let's see how this is done.

- Let's dive deeper into AJAX by **creating a dynamic weather information page**.
- The user will input the location for which they want weather information.
- When the user submits the location, an AJAX request will be sent to the weather API.
- The response from the API, containing the weather data, will be received.
- The response will be parsed, extracting the relevant weather information.
- Finally, the parsed weather information will be displayed to the user on the page.

Weather Forecast

Select the location you want:

Kindly add Longitude and Latitude information

Weather Forecast

Select the location you want:

City: Jeddah

Temperature: 31.97°C

Feels Like: 34.31°C

Wind Speed: 0.96°C

Description: clear sky

APIs, Webservices, and AJAX

Let's see how this is done.

- HTML Code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Web Services Assignment</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Weather Forecast</h1>
    <p>Select the location you want:</p>
    <input type="text" placeholder="longitude" name="long" />
    <input type="text" placeholder="latitude" name="lat" />
    <br /><br />
    <button onclick="getData()">Get Weather Data</button>
    <button>Clear Wather Data</button>
    <div id="weatherData"></div>
    <script src="script.js"></script>
  </body>
</html>
```

Weather Forecast

Select the location you want:

longitude	latitude
Get Weather Data	Clear Wather Data

Don't forget the event handler!

APIs, Webservices, and AJAX

Sending AJAX w/ native JS.

- JS Code using native JS:

```
function getData() {  
    const xhr = new XMLHttpRequest();  
    // xhr.open('GET', 'API_ENDPOINT', true);  
    const key = "46c19abd256cf11480ebcbc3a69585fd";  
    // let lat= 44.34;  
    // let lon = 10.99;  
    let lat = document.getElementsByName("long")[0].value;  
    let lon = document.getElementsByName("lat")[0].value;  
  
    if (lat.match("[0-9]+.[0-9]*")) {  
        let endpoint = `https://api.openweathermap.org/data/2.5/weather?  
units=metric&lat=${lat}&lon=${lon}&appid=${key}`;  
        xhr.open("GET", endpoint, true);  
        xhr.onload = function () {  
            if (xhr.status === 200) {  
                const response = JSON.parse(xhr.responseText);  
                displayData(response);  
            } else {  
                displayError("Failed to fetch weather data.");  
            }  
        };  
        xhr.onerror = function () {  
            displayError("An error occurred while fetching weather data.");  
        };  
        xhr.send();  
    } else {  
        displayError("Kindly add Longitude and Latitude information");  
    }  
}
```

Weather Forecast

Select the location you want:

Get Weather Data

Clear Wather Data

Kindly add Longitude and Latitude information

Validating user input to be numbers.

Preparing link by adding requested parameters as per the API

Sending request and handling response

APIs, Webservices, and AJAX

We can also do the same thing use the `fetch()` API as follows

- Sending AJAX using the fetch API:

```
async function getData() {
  const key = "46c19abd256cf11480ebcbc3a69585fd";
  let lat = document.getElementsByName("long")[0].value;
  let lon = document.getElementsByName("lat")[0].value;
  const endpoint = `https://api.openweathermap.org/data/2.5/
weather?units=metric&lat=${lat}&lon=${lon}&appid=${key}`;
  console.log(endpoint);

  if (lat.match("[0-9]+.[0-9]*")) {
    try{
      const response = await fetch(endpoint)
      const data = await response.json()
      displayData(data);
    }catch(err){
      displayError("An error occured while fetching weather data.");
    }
  } else {
    displayError("Kindly add Longitude and Latitude information");
  }
}
```

Sending
request

Handling
errors

APIs, Webservices, and AJAX

Parsing JSON:

- JS Code:

```
function displayData(data) {  
  const weatherDataElement = document.getElementById("weatherData");  
  weatherDataElement.innerHTML = "";  
  
  const cityName = data.name;  
  const temperature = data.main.temp;  
  const description = data.weather[0].description;  
  const feelsLike = data.main.feels_like;  
  const windSpeed = data.wind.speed;  
  
  const cityElement = document.createElement("p");  
  cityElement.textContent = `City: ${cityName}`;  
  
  const temperatureElement = document.createElement("p");  
  temperatureElement.textContent = `Temperature: ${temperature}°C`;  
  
  const feelsLikeElement = document.createElement("p");  
  feelsLikeElement.textContent = `Feels Like: ${feelsLike}°C`;  
  
  const windElement = document.createElement("p");  
  windElement.textContent = `Wind Speed: ${windSpeed}°C`;  
  
  const descriptionElement = document.createElement("p");  
  descriptionElement.textContent = `Description: ${description}`;  
  
  weatherDataElement.appendChild(cityElement);  
  weatherDataElement.appendChild(temperatureElement);  
  weatherDataElement.appendChild(feelsLikeElement);  
  weatherDataElement.appendChild(windElement);  
  weatherDataElement.appendChild(descriptionElement);  
}  
  
function displayError(errorMessage) {  
  const weatherDataElement = document.getElementById("weatherData");  
  weatherDataElement.innerHTML = "";  
  
  const errorElement = document.createElement("p");  
  errorElement.classList.add("error");  
  errorElement.textContent = errorMessage;  
  
  weatherDataElement.appendChild(errorElement);  
}
```

Showing weather
information send by the API

Weather Forecast

Select the location you want:

21.48 39.19

Get Weather Data

Clear Wather Data

City: Jeddah
Temperature: 31.97°C
Feels Like: 34.31°C
Wind Speed: 0.96°C
Description: clear sky

Showing errors



Any questions?
Please feel free to raise your
hands and ask.